

第八章：并发

引言

本章导读

到本章开始之前，我们已经完成了组成应用程序执行环境的操作系统的三个重要抽象：进程、地址空间和文件。操作系统基于处理器的时间片不断切换进程，从而在宏观上实现多个应用程序的并发执行，但这种并发仅限于不同进程之间。

接下来，我们将引入线程（Thread）：它的作用是提高单个进程内部的并发能力。

为什么已经有了进程，还需要线程呢？

这是因为，很多单进程应用程序在逻辑上其实包含多个可以并行执行的任务。如果没有多线程机制，当其中一个任务被阻塞时，即使其他任务与它没有依赖关系，也会被迫一起等待，导致整体效率下降。

举个例子：我们在使用 Word 时，通常会开启“定时自动保存”功能。这样，即使电脑突然崩溃或停电，文档内容也可能已经被提前保存下来。

假设 Word 没有多线程支持，那么在执行自动保存操作时，如果磁盘写入速度较慢（比如机械硬盘繁忙），整个 Word 进程就可能被操作系统暂时挂起。对我们用户来说，就会感觉 Word 隔一会儿就卡顿一下，严重影响使用体验。

因此，引入线程可以让一个进程内的多个任务独立运行、互不干扰，从而提升程序的响应性和整体性能。

线程定义

简单地说，线程是进程的组成部分。一个进程可以包含 1 到 n 个线程。属于同一个进程的所有线程共享该进程的资源，比如地址空间、打开的文件等。

一个基本的线程由以下几部分组成：

1. 线程 ID
2. 执行状态
3. 当前指令指针（PC）
4. 寄存器集合
5. 栈

线程是可以被操作系统或用户态调度器独立调度（Scheduling）和分派（Dispatch）的基本单位。

在本章之前，我们把进程看作程序的基本执行实体——即程序在某个数据集合上的一次运行活动，同时也是系统进行资源（如处理器、地址空间和文件等）分配与调度的基本单位。

引入线程之后，这个定义需要调整：现在，进程主要作为线程的资源容器，而线程则成为程序真正的基本执行实体。

同步互斥

在前面提到了同步、互斥和数据一致性，它们到底是什么意思呢？

当多个线程共享同一个进程的地址空间时，每个线程都可以访问该进程中的数据（例如全局变量）。

1. 如果每个线程所使用的变量都是其他线程不会读取或修改的，那么就不会出现数据一致性问题。
2. 如果变量是只读的，即使多个线程同时读取它，也不会引发一致性问题。

但是，当一个线程正在修改某个变量，而另一个线程同时读取这个变量时，就可能读到一个“中间状态”或“不完整”的值，从而导致程序行为异常。这就是所谓的数据不一致性问题。

注解

并发相关术语

1. 共享资源 (shared resource)：指可以被多个线程或进程共同访问的变量或数据结构。
2. 临界区 (critical section)：指程序中访问共享资源的那一段代码。
3. 竞态条件 (race condition)：当多个线程或进程同时进入临界区，并试图修改同一共享数据结构时，最终结果依赖于它们执行的相对顺序，从而产生错误或不可预期的行为。
4. 不确定性 (indeterminate)：由于存在竞态条件，程序的执行结果会随着线程调度的时机不同而变化，无法稳定复现，而开发者通常希望程序行为是确定的。
5. 互斥 (mutual exclusion)：一种机制或操作原语，用于确保在任意时刻，最多只有一个线程能够进入临界区，从而避免竞态条件，保证结果的确定性。
6. 原子性 (atomic)：指一组操作要么全部完成，要么完全不发生，不会停留在中间状态。在数据库中，具有原子性的一组操作被称为事务 (transaction)。
7. 同步 (synchronization)：指多个并发执行的线程或进程在某些关键点上需要协调彼此的执行顺序，比如等待对方完成某项操作，这种协调关系称为同步。
8. 死锁 (dead lock)：指一组线程或进程中的每一个都在等待只有该组中其他成员（包括自己）才能触发的事件，结果谁也无法继续执行，系统陷入停滞。
9. 饥饿 (hungry)：指某个线程或进程虽然具备运行条件，但由于调度策略或其他原因，长期得不到执行机会，仿佛被“遗忘”了。

在后续章节中，这些术语会被频繁使用。如果你现在对它们的理解还不够清晰，不用着急。随着后续内容的逐步展开和实验的深入，你会逐渐掌握这些概念的实际含义和应用场景。

实践体验

获取本章代码：

```
$ git clone https://github.com/LearningOS/uCore-Tutorial-Code-2023S.git
$ cd uCore-Tutorial-Code-2023S
$ git checkout ch8
$ git clone https://github.com/LearningOS/uCore-Tutorial-Test-2023S.git user
```

或者你也可以在自己原来的仓库里 fetch 它，记得更新测例仓库的代码。

在 qemu 模拟器上运行本章代码：

```
$ make BASE=1 test
```

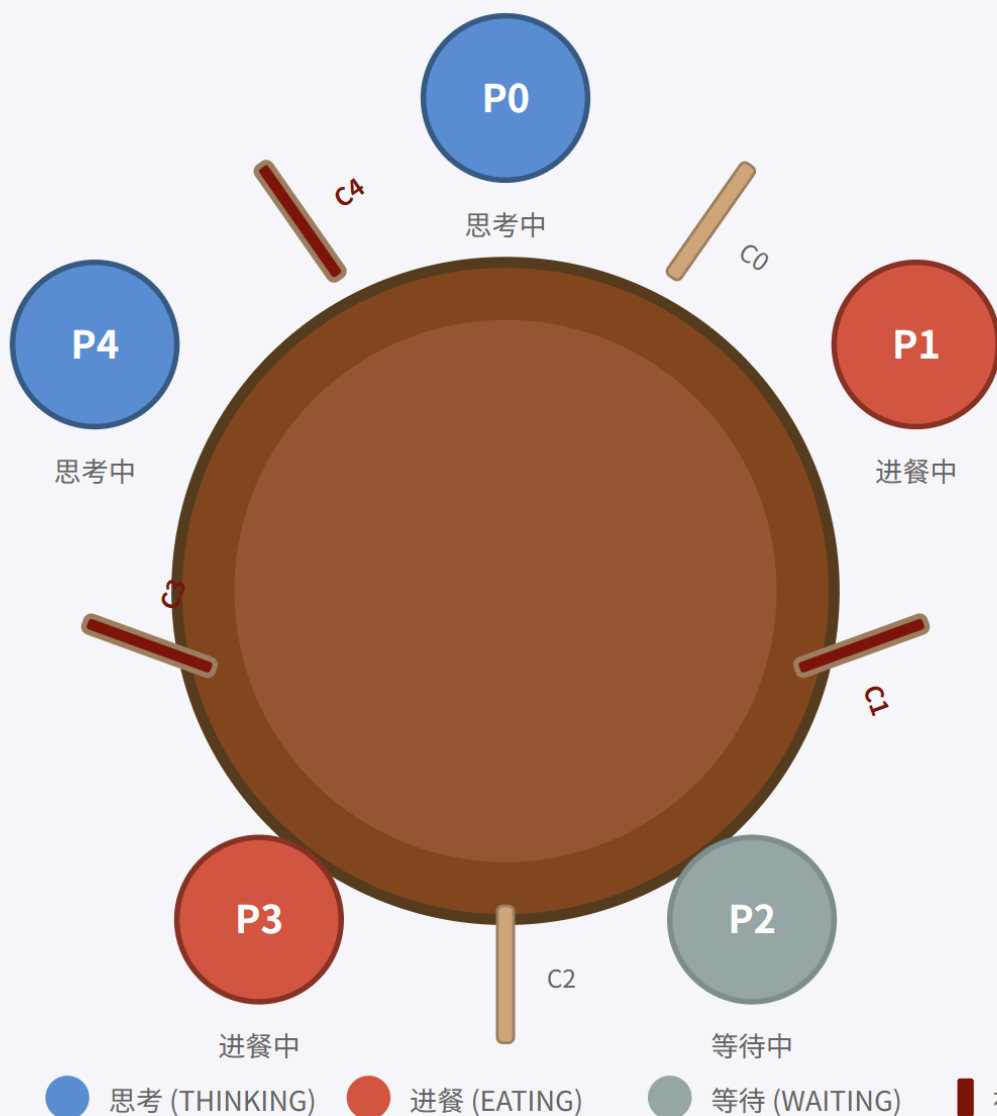
内核初始化完成之后就会进入 shell 程序，我们可以体会一下线程的创建和执行过程。在这里我们运行一下本章的测例 ch8b_threads：

```
>> ch8b_threads
aaa...bbb...ccc...
thread#1 exited with code 1
thread#2 exited with code 2
thread#3 exited with code 3
threads test passed!
Shell: Process 2 exited with code 0
>>
```

它会有4个线程在执行，等前3个线程执行完毕并输出大量 a/b/c 后，主线程退出，导致整个进程退出。

此外，在本章的操作系统支持通过互斥来执行"哲学家就餐问题"这个应用程序：

哲学家就餐问题 (Dining Philosophers)



规则: 哲学家必须同时拿到左右两根筷子才能进餐

```
>> ch8b_mut_phi_din
Here comes 5 philosophers!
Phil threads created
```

```
time cost = 720 ms
'- ' -> THINKING; 'x' -> EATING; ' ' -> WAITING

#0: -----          XXXXXXXX-----          XXXX----- XXXXXX---
#1: ----XXXXX---   XXXXXXXX-----          X----XXXXXX
#2: -----          XX-----X-----XXXXX-----          XXXXX
#3: -----XXXXXXXXX-----XXXX-----          XXXXXX--- XXXXXXXXXXXX
#4: -----          X-----          XXXXXX--- XXXXX----- XXX

Shell: Process 2 exited with code 0
>>
```

图例说明：

符号	状态	说明
-	THINKING	哲学家正在思考
x	EATING	哲学家正在进餐（持有两根筷子）
(空格)	WAITING	哲学家正在等待筷子

从上面的输出可以观察到：

- 1. **互斥性**：任意时刻，相邻的两位哲学家不会同时进餐（因为他们共享一根筷子）
- 2. **无死锁**：所有哲学家最终都能进餐，没有出现全部阻塞的情况
- 3. **无饥饿**：每位哲学家都有机会进餐，不会被永久排斥

注解

哲学家就餐问题

计算机科学家 Dijkstra 提出并研究的“哲学家就餐问题”是一个经典的进程（或线程）同步与互斥问题。该问题的描述如下：

有 5 个哲学家围坐在一张圆桌旁，每人坐在一张椅子上。圆桌上放着 5 个碗和 5 只筷子，每两只相邻的哲学家之间放一只筷子（即每位哲学家左右各有一只筷子）。

每位哲学家的生活方式是交替进行“思考”和“进餐”：

- 1. 当哲学家在思考时，不占用任何资源；
- 2. 当他感到饥饿时，会尝试拿起自己左边和右边的两只筷子；
- 3. 只有当他同时拿到两只筷子后，才能开始进餐；
- 4. 进餐结束后，他会把两只筷子放回原处，继续思考。

这个问题的核心在于：如何设计一种机制，使得所有哲学家都能顺利进餐，而不会出现死锁（例如所有哲学家同时拿起左边的筷子，然后都在等待右边的筷子）或饥饿（某个哲学家长时间无法获得筷子）的情况。

它被广泛用于演示并发控制中的同步、互斥、死锁和资源分配等关键概念。

本章代码树

- 1.

```
2|— bootloader
3|  └─ rustsbi-qemu.bin
4|— LICENSE
5|— Makefile
6|— nfs
7|  └─ fs.c
8|  └─ fs.h
9|  └─ Makefile
10|  └─ types.h
11|— os
12|  └─ bio.c
13|  └─ bio.h
14|  └─ console.c
15|  └─ console.h
16|  └─ const.h
17|  └─ defs.h
18|  └─ entry.S
19|  └─ fcntl.h
20|  └─ file.c
21|  └─ file.h
22|  └─ fs.c
23|  └─ fs.h
24|  └─ kalloc.c
25|  └─ kalloc.h
26|  └─ kernel.ld
27|  └─ kernelld.py
28|  └─ kernelvec.S
29|  └─ loader.c (修改: 更改了加载用户程序的逻辑, 此时不再为进程分配用户栈)
30|  └─ loader.h
31|  └─ log.h (修改: log头中新增了线程号打印)
32|  └─ main.c
33|  └─ pipe.c
34|  └─ plic.c
35|  └─ plic.h
36|  └─ printf.c
37|  └─ printf.h
38|  └─ proc.c (修改: 为每个线程而非进程分配栈空间和trapframe; 更改进程初始化逻辑, 为其分配主线程; 任务调度粒度从进程改为线程; 新增线程id与线程指针转换的辅助函数; 新增线程分配、释放逻辑; exit由退出进程改为退出线程)
39|  └─ proc.h (修改: 新增线程相关结构体和状态枚举; 在PCB中新增线程相关变量; 增改部分函数签名)
40|  └─ queue.c (修改: 由进程专用队列改为通用队列, 初始化时需指定数组地址和大小)
41|  └─ queue.h (修改: 同 queue.c)
42|  └─ riscv.h
43|  └─ sbi.c
44|  └─ sbi.h
45|  └─ string.c
46|  └─ string.h
47|  └─ switch.S
48|  └─ sync.c (新增: 实现了mutex、semaphore、condvar相关操作)
49|  └─ sync.h (新增: 声明了mutex、semaphore、condvar相关操作)
50|  └─ syscall.c (修改: 增加 sys_thread_create、sys_gettid、sys_waittid 以及三种同步互斥结构所使用的系统调用)
51|  └─ syscall.h (修改: 同syscall.c)
```

```
52|  └─ syscall_ids.h (修改: 为新增系统调用增加了调号号的宏定义)
53|  └─ timer.c
54|  └─ timer.h
55|  └─ trampoline.S
56|  └─ trap.c (修改: 将进程trap改为线程trap, 新增用户态虚存映射辅助函数uvmmmap)
57|  └─ trap.h (修改: 同trap.c)
58|  └─ types.h
59|  └─ virtio_disk.c
60|  └─ virtio.h
61|  └─ vm.c
62|  └─ vm.h
63| └─ README.md
64| └─ scripts
65|   └─ initproc.py
```

内核态的线程管理

线程概念

这里会结合与进程的比较来说明线程的概念。到本章之前，我们看到了进程这一抽象：操作系统让进程拥有相互隔离的虚拟地址空间，并让每个进程感觉自己独占一个虚拟处理器。其实这只是操作系统通过时分复用（时间上轮流使用 CPU）和空分复用（空间上划分内存）技术，让多个进程共享有限的物理内存和物理 CPU。

而线程是在进程内部引入的一种新抽象。在没有线程之前，一个进程在任意时刻只有一个执行点（也就是程序计数器 PC 寄存器中保存的下一条要执行指令的地址）。线程的引入将这个单一执行点扩展为多个执行点——也就是说，一个进程内可以同时存在多个线程，每个线程都有自己的执行位置。更重要的是，这些线程共享所属进程的地址空间，因此它们可以直接访问进程内的数据，而不需要使用相对复杂的进程间通信（IPC）机制（这类机制通常需要内核介入）。

举个例子：如果两个线程想交换数据，它们可以直接读写同一个全局变量；但如果是两个进程，就需要通过管道、消息队列等 IPC 方式，效率更低且更复杂。

在线程实际运行时，它需要：

1. 程序计数器（PC）寄存器，记录当前执行到哪条指令；
2. 一组通用寄存器，用于存放当前指令操作的数据；
3. 一个独立的栈，用来保存函数调用记录、局部变量等信息。

这三部分构成了线程上下文的核心内容。

当两个线程运行在同一个处理器上时，操作系统必须像调度进程一样，在适当的时候切换线程，并保存/恢复各自的上下文。这样一来，同一进程中的多个线程就可以“看似同时”地独立运行。也正因为如此，线程取代了进程，成为现代操作系统调度的基本单位。

由于引入了线程，进程的结构被进一步细化：线程负责表示对处理器的虚拟化，而进程则退化为管理线程的“容器”。在一个进程内部，所有线程之间没有严格的父子关系，可以看作是“兄弟”关系，但仍然有一个“老大”。这个“老大”线程，就是在创建进程时（例如通过 fork 系统调用）自动建立的第一个线程，它的线程标识符（TID）为 0。

线程模型与重要系统调用

目前，我们只介绍本章所实现的内核中采用的一种非常简单的线程模型。该模型具有以下特点：

1. 线程有三种运行状态：就绪态、运行态和等待态；
2. 所有线程共享所属进程的地址空间及其他资源（如打开的文件等）；
3. 线程可被操作系统调度，分时占用 CPU 执行；
4. 线程可以动态创建和退出；
5. 线程可通过系统调用请求操作系统提供服务。

我们实现的线程模型建立在进程的地址空间抽象之上：每个线程共享进程的代码段、全局数据段、堆等可共享区域，但各自拥有独立的栈空间。

为了支持灵活的多线程应用，操作系统需要提供一些关键的系统调用，例如：

1. 创建新线程；
2. 等待子线程结束。

接下来将介绍这些系统调用的基本功能和设计思路。

线程创建系统调用

在一个进程运行过程中，它可以创建多个属于自己的线程。每个线程都有唯一的线程标识符（TID，Thread Identifier）。

系统调用 `thread_create` 的原型如下：

```
1/// 功能：当前进程创建一个新的线程
2/// 参数：entry 表示线程的入口函数地址
3/// 参数：arg：表示线程的一个参数
4int sys_thread_create(uint64 entry, uint64 arg)
```

当进程调用 `thread_create` 系统调用后，内核会在该进程内部创建一个新的线程。这个新线程可以访问进程所拥有的代码段、堆以及其他数据段，因为它们属于同一个地址空间。但为了保证线程能够独立运行，内核会为此新线程分配一个专属的用户态栈。

注意：每个线程必须有自己的栈，否则函数调用、局部变量等就会相互干扰，导致程序行为不可预测。

此外，由于用户态和内核态使用不同的页表（即地址空间不直接共享），系统需要借助一个特殊的“跳板页”（TRAMPOLINE）来实现从用户态到内核态的平滑切换。因此，当进程中存在多个线程时，每个线程都需要自己独立的 TRAMPOLINE 页，用于处理自身在发生系统调用或异常时的地址空间转换。

与创建进程的 `fork` 系统调用相比，创建线程最大的区别在于：**不需要建立新的地址空间**。`fork` 会复制整个进程的地址空间，而 `thread_create` 只是在已有地址空间内新增一个执行流。

另外，同一进程中的线程之间没有父子关系——它们是平等的“兄弟”线程，这与进程模型中常见的父子进程关系完全不同。

等待子线程系统调用

当一个线程完成其任务后，会通过 `exit` 系统调用主动退出。内核在接收到该调用后，会立即回收该线程占用的部分资源，主要包括用户态相关的资源，例如：

1. 用户态栈；

2. 用于系统调用和异常处理的跳板页（TRAMPOLINE）等。

但注意：线程在内核态使用的资源（如内核栈、线程控制块等）并不会在 `exit` 时立刻释放。

这些内核态资源需要由进程的主线程（或其他线程）显式调用 `waittid` 系统调用来回收。只有在 `waittid` 成功返回后，该线程的所有资源才被彻底释放，线程才算真正“销毁”。

系统调用 `waittid` 的原型如下：

```
1/// 参数: tid表示线程id
2/// 返回值: 如果线程不存在, 返回-1; 如果线程还没退出, 返回-2; 其他情况下, 返回结束线程的退出码
3int sys_waittid(int tid);
```

一般情况下，进程（或主线程）需要负责调用 `waittid` 来等待它所创建的其他线程（即非主线程）结束，并回收这些线程在内核中占用的资源，例如：

1. 线程的内核栈；
2. 线程控制块（TCB）等数据结构。

为什么要显式回收？因为如果只调用 `exit` 而不调用 `waittid`，内核资源会“泄漏”——虽然线程不再运行，但其内核数据仍占据内存，长期积累会导致系统资源耗尽。

但如果进程（或主线程）先调用了 `exit` 系统调用退出，那么整个进程（包括该进程内的所有线程）都会被强制终止。此时，该进程的父进程可以通过 `waitpid` 系统调用来回收这个已退出子进程尚未释放的剩余资源（包括那些未被 `waittid` 回收的线程内核资源）。

进程相关的系统调用

在引入线程机制后，原有的三个重要进程系统调用 —— `fork`、`exec` 和 `waitpid` —— 在接口上保持不变，但在内部实现上需要进行一定扩展。

首先需要明确：原来属于“进程”的处理器执行上下文（如 PC、寄存器、栈等）现在被拆分到了“线程”中。因此：

- 当通过 `fork` 创建新进程时，内核不仅要复制地址空间，还要为新进程创建一个初始的主线程（TID 为 0），并初始化用于管理后续线程的线程控制块向量。
- 相比之下，`exec` 和 `waitpid` 的改动较小，基本沿用原有的进程处理逻辑。

总体而言，这三个系统调用仍然保持原有的进程级语义：`fork` 用于创建新进程，`exec` 用于加载新程序，`waitpid` 用于等待子进程结束。线程的引入并未改变它们对外的行为，只是内部实现更复杂了。

应用程序示例

我们刚刚介绍了 `thread_create` 和 `waittid` 这两个关键的系统调用。结合之前已实现的系统调用，就可以编写功能更灵活的多线程应用程序。下面通过一个名为 `threads` 的多线程程序的开发过程，来演示如何使用这些系统调用。

系统调用封装

同学们可以在 `user/lib/syscall.c` 文件中看到这些系统调用的封装代码：

```
1int thread_create(void *entry, void *arg)
2{
3    // 首次创建线程时启用, 此时必须是单线程状态
4    if (!buffer_lock_enabled) {
5        enable_thread_io_buffer();
```



```

6   }
7   return syscall(SYS_thread_create, (uint64)entry, (uint64)arg);
8}
9
10int waittid(int tid)
11{
12   int ret = syscall(SYS_waittid, tid);
13   while (ret == -2) {
14       sched_yield();
15       ret = syscall(SYS_waittid, tid);
16   }
17   return ret;
18}

```

waittid 用于等待一个线程标识符 (TID) 为 tid 的线程结束。在具体实现中, 当 sys_waittid 的返回值为 -2 时, 表示目标线程确实存在, 但尚未退出。此时, 主线程会调用 yield_ 主动放弃 CPU 使用权, 让出给其他线程运行; 等到下一次调度器重新将 CPU 分配给它时, 再重新调用 sys_waittid 检查目标线程是否已经退出。

这种“轮询 + 主动让出 CPU”的方式虽然不是最高效的 (理想情况是内核提供阻塞等待机制), 但它能显著减少空转对 CPU 资源的浪费, 同时避免了在内核中实现复杂的线程等待队列, 从而简化了内核的设计。

特别需要注意的是, 在 thread_create 的实现中有一段对全局变量 buffer_lock_enabled 的判断。这是因为我们的标准输出使用了输出缓冲区, 而早期版本的缓冲区并未考虑多线程安全问题。为了在多线程环境下正确使用输出功能, 我们引入了一个互斥锁来保护缓冲区的访问。这个线程安全模式会在主线程第一次调用 thread_create 时被自动启用——即设置 buffer_lock_enabled 为 true, 并初始化相应的锁。

多线程应用程序 – threads

多线程应用程序 threads 启动后, 首先调用 thread_create 创建三个新线程。加上进程自带的主线程, 此时共有四个线程并发执行。

每个线程 (包括主线程之外的三个) 会循环打印 1000 个字符, 然后调用 exit 系统调用主动退出。主线程则通过三次调用 waittid, 依次等待这三个子线程全部结束。只有当所有子线程都退出并被成功回收后, 主线程才会结束整个进程的执行。

下面是多线程应用程序 threads 的源代码:

```

1//user/src/ch8b_threads.c
2...
3#define LOOP 1000
4#define NTHREAD 3
5
6void thread_a()
7{
8   for (int i = 0; i < LOOP; ++i) {
9       putchar('a');
10  }
11  exit(1);
12}
13
14void thread_b()
15{
16   for (int i = 0; i < LOOP; ++i) {

```

```

17     putchar('b');
18 }
19 exit(2);
20}
21
22void thread_c()
23{
24     for (int i = 0; i < LOOP; ++i) {
25         putchar('c');
26     }
27     exit(3);
28}
29
30int main(void)
31{
32     int tids[NTHREAD];
33     tids[0] = thread_create(thread_a, 0);
34     tids[1] = thread_create(thread_b, 0);
35     tids[2] = thread_create(thread_c, 0);
36     for (int i = 0; i < NTHREAD; ++i) {
37         int tid = tids[i];
38         int exit_code = waittid(tid);
39         printf("thread %d exited with code %d\n", tid, exit_code);
40         assert_eq(tid, exit_code);
41     }
42     puts("threads test passed!");
43     return 0;
44}

```

线程管理的核心数据结构

为了在现有进程管理的基础上实现线程管理，我们需要对一些数据结构的内容和接口进行改进。基本思路是：把进程中与处理器相关的部分拆分出来，单独形成线程相关的部分。

本节将按照如下顺序来进行介绍：

1. 线程控制块 thread：表示线程的核心数据结构。
2. 调度器 scheduler：用于线程调度。

线程控制块

在内核中，每个线程的执行状态、线程上下文等信息都保存在一个被称为“线程控制块”的结构中。这个结构是内核用来管理线程的核心数据结构。在内核看来，一个线程控制块就等价于一个线程。

什么是“线程上下文”？它主要包括线程运行时所需的寄存器值、程序计数器（PC）、栈指针等信息。当线程被切换出去时，这些信息会被保存在线程控制块中；当线程再次被调度运行时，系统会从这里恢复上下文，使线程看起来像是从未中断过一样。

```

1 struct thread {
2     enum threadstate state; // 线程状态
3     int tid; // 线程ID
4     struct proc *process;
5     uint64 ustack; // 用户栈虚拟地址
6     uint64 kstack; // 内核栈虚拟地址
7     struct trapframe *trapframe; // trampoline.S使用的数据页
8     struct context context; // 切换到此处运行进程
9     uint64 exit_code;
10};

```

之前进程中的定义不存在的：

1. tid：线程标识符
2. process：线程所属的进程

与之前进程中的定义相同/类似的部分：

1. trapframe：线程的 Trap 上下文在内核中的地址。
2. context：保存暂停线程的线程上下文，用于线程切换。
3. state：维护当前线程的执行状态。
4. exit_code：线程退出码。

包含线程的进程控制块

把线程相关数据单独组织成数据结构后，进程的结构也需要进行一定的调整：

```

1 // 进程状态
2 struct proc {
3     ...
4     uint64 ustack_base; // 用户栈基地址的虚拟地址
5     struct thread threads[NTHREAD];
6 };

```

从中可以看出，进程把与处理器执行相关的部分都移到了 thread 中，并组织为一个线程控制块数组，这就自然对应到多个线程的管理上了。此外，由于多个线程拥有各自不同的用户栈空间，需要在进程控制块中记录用户栈的地址，每个线程的用户栈基地址为 $ustack_base + tid * USTACK_SIZE$ 。

例如，若 USTACK_SIZE 是 4KB，那么 tid 为 0 的线程使用从 ustack_base 开始的 4KB 空间，tid 为 1 的线程则使用从 $ustack_base + 4KB$ 开始的下一个 4KB 空间，以此类推。这样可以避免不同线程的用户栈相互覆盖。

线程管理机制的设计与实现

在上述线程模型和内核数据结构的基础上，我们还需完成线程管理的基本实现，从而构造出一个完整的“达科塔盗龙”操作系统。本节将分析如何实现线程管理：

1. 线程创建、线程退出与等待线程结束
2. 线程执行中的特权级切换

线程创建、线程退出与等待线程结束

线程创建

当一个进程执行中发出了创建线程的系统调用 `sys_thread_create` 后，操作系统就需要在当前进程的基础上创建一个线程了。这里重点是需要了解初始化线程控制块各个成员变量，建立好进程和线程的关系等。只有建立好这些成员变量，才能给线程建立一个灵活方便的执行环境。这里列出支持线程正确运行所需的重要的执行环境要素：

1. 线程的用户态栈：确保在用户态的线程能正常执行函数调用；
2. 线程的内核态栈：确保线程陷入内核后能正常执行函数调用；
3. 线程的跳板页：确保线程能正确地进行用户态 ↔ 内核态切换；
4. 线程上下文：即线程用到的寄存器信息，用于线程切换。

线程创建的具体实现如下：

```
1// os/syscall.c
2int sys_thread_create(uint64 entry, uint64 arg)
3{
4    struct proc *p = curr_proc();
5    int tid = allocthread(p, entry, 1);
6    if (tid < 0) {
7        errorf("fail to create thread");
8        return -1;
9    }
10   struct thread *t = &p->threads[tid];
11   t->trapframe->a0 = arg;
12   t->state = RUNNABLE;
13   add_task(t);
14   return tid;
15}
```

上述代码主要完成了如下事务：

1. 第 4 行，找到当前正在执行的线程所属的进程 `p`。
2. 第 5 行，调用 `allocthread` 函数，创建一个新的线程，线程编号 `tid` 即为该函数的返回值。在创建过程中，建立与进程 `p` 的所属关系，分配了线程用户态栈、内核态栈、用于异常/中断的跳板页，设置线程的函数入口点、任务上下文、内核栈和用户栈地址。
3. 第 11-12 行，设置线程的传入参数，并将其状态设为可被调度运行。
4. 第 13 行，把线程挂到调度队列中。

线程退出

当一个非主线程的其他线程发出 `sys_exit` 系统调用时，内核会调用 `exit` 函数退出当前线程并切换到下一个线程，但不会导致其所属进程的退出。当主线程（即进程的初始线程）发出这个系统调用时，内核会回收整个进程（这包括了其管理的所有线程）的资源，并退出。具体实现如下：

注意区分“线程退出”和“进程退出”：非主线程退出只释放自身资源，而主线程退出意味着整个进程结束，所有属于该进程的线程都会被强制终止并清理。这是因为在大多数操作系统模型中，主线程代表了整个进程的生命周期。

```
1// os/proc.c
```

```

2void exit(int code)
3{
4    struct proc *p = curr_proc();
5    struct thread *t = curr_thread();
6    t->exit_code = code;
7    t->state = EXITED;
8    int tid = t->tid;
9    debugf("thread exit with %d", code);
10   freethread(t);
11   if (tid == 0) {
12       p->exit_code = code;
13       freeproc(p);
14       debugf("proc exit");
15       if (p->parent != NULL) {
16           // 父进程应该调用 `wait`
17           p->state = ZOMBIE;
18       }
19       // 将所有子进程的 `parent` 设为 NULL
20       struct proc *np;
21       for (np = pool; np < &pool[NPROC]; np++) {
22           if (np->parent == p) {
23               np->parent = NULL;
24           }
25       }
26   }
27   sched();
28}

```

上述代码主要完成了如下事务：

1. 第 10 行，调用 freethread 回收线程的各种资源。
2. 第 11-26 行，如果是主线程发出的退出请求，则回收整个进程的部分资源，并退出进程。其中第 20-25 行所做的事情是将当前进程的所有子进程的父进程置为 NULL。
3. 第 27 行，进行线程调度切换。

上述实现中很大一部分与第五章讲解的“进程的退出”的功能实现大致相同。

这里之所以要将子进程的父进程设为 NULL，是因为当前进程即将退出，不能再作为其他进程的父进程。这是为了防止出现“孤儿进程”无法被正确回收的问题（尽管在更完整的系统中，通常会由 init 进程接管，但在本实验简化模型中直接置 NULL）。

等待线程结束

主线程通过系统调用 sys_waittid 来等待其他线程的结束。具体实现如下：

```

1// os/syscall.c
2int sys_waittid(int tid)
3{
4    if (tid < 0 || tid >= NTHREAD) {
5        errorf("unexpected tid %d", tid);
6        return -1;
7    }
8    struct thread *t = &curr_proc()->threads[tid];

```

```

9  if (t->state == T_UNUSED || tid == curr_thread()->tid) {
10     return -1;
11 }
12 if (t->state != EXITED) {
13     return -2;
14 }
15 memset((void *)t->kstack, 7, KSTACK_SIZE);
16 t->tid = -1;
17 t->state = T_UNUSED;
18 return t->exit_code;
19}

```

上述代码主要完成了如下事务：

1. 第 4-11 行，如果是线程等待一个不存在的 tid 或等待自己，则返回错误。
2. 第 12-14 行，如果 tid 对应的线程还未退出，则返回错误。
3. 第 16-18 行，对应的线程已经退出，则将该线程的线程控制块标记为已释放，并返回该线程的退出码。

注意：sys_waittid 只能用于等待已经结束的线程（即“非阻塞式”等待）。如果被等待的线程还在运行，调用会立即失败，而不是挂起当前线程直到目标线程结束。这与某些系统中 waitpid 的阻塞行为不同，是本实验设计的简化之处。

线程执行中的特权级切换和调度切换

线程执行时的特权级切换过程与第三章“多道程序与协作式调度”部分所述机制相符，而线程执行过程中的调度切换则遵循第五章“进程管理的核心数据结构”一节所阐述的流程。

锁机制

本节导读

到目前为止，我们已经实现了进程和线程，并且理解了在一段时间内，可能会有多个线程同时处于执行状态，这就是所谓的“并发”。

由于引入了线程，多个线程可以共享同一个进程中的全局数据。当多个线程同时尝试读取或修改这些共享数据时，谁先完成更新，完全取决于操作系统内核采用的抢占式调度与任务分派策略。

操作系统会根据时间片、优先级等因素，在多个线程之间快速切换。这种切换是透明的，对程序员来说不可预测。因此，即使你写的代码看起来是“顺序”执行的，实际运行时可能被随时打断。

在一般情况下，任何一个线程都有可能先执行；而且，由于中断等外部事件，线程的执行可能在任意时刻被暂停，转而执行另一个线程。这就导致多个线程在一段时间内交替执行。

如果没有专门的保障机制（例如互斥、同步等手段），那么这些交替执行、同时访问共享数据的线程，很可能无法得到预期的正确结果。

因此，我们需要引入一种保障机制——锁。锁的作用是：无论操作系统如何抢占、调度或切换线程，只有持有锁的线程才能独占式地对共享数据进行读写操作，从而确保结果的正确性。

锁的实现依赖于处理器提供的特殊指令（如原子操作）以及操作系统提供的系统调用支持。这些底层机制共同保证了多个线程在访问共享资源时能够互斥执行。

接下来的几个小节将依次讲解：为什么需要锁、锁的基本设计思路，以及锁的不同实现方式。

为什么需要锁

上一小节已经提到：如果没有适当的保护机制，多个线程在并发读写共享数据时，可能无法得到预期的结果。下面我们通过一个简单的例子来说明这个问题：

```
1// 线程入口函数
2int a=0;
3void f() {
4    a = a + 1;
5}
```

对于上述函数中的第 4 行代码，一般人理解处理器会一次就执行完这条简单的语句，但实际情况并不是这样。我们可以用 GCC 编译出上述函数的汇编码：

```
1$ riscv64-unknown-elf-gcc -o f.s -S f.c
```

可以看到生成的汇编代码如下：

```
1//f.s
2    .text
3    .globl    a
4    .section    .sbss,"aw",@nobits
5    .align    2
6    .type      a, @object
7    .size      a, 4
8a:
9    .zero      4
10   .text
11   .align      1
12   .globl      f
13   .type      f, @function
14f:
15   addi        sp,sp,-16
16   sd          s0,8(sp)
17   addi        s0,sp,16
18   lui         a5,%hi(a)
19   lw          a5,%lo(a)(a5)
20   addiw        a5,a5,1
21   sext.w      a4,a5
22   lui         a5,%hi(a)
23   sw          a4,%lo(a)(a5)
24   nop
25   ld          s0,8(sp)
26   addi        sp,sp,16
27   jr          ra
```

从中可以看出，高级语言中看似简单的一条语句（例如 C 代码的第 4 行，对全局变量进行读写操作），在底层实际上可能被编译为多条汇编指令（对应汇编代码的第 18~23 行）。

如果这个函数会被多个线程同时调用，那么在执行上述汇编代码的过程中——也就是从第 18 行到第 23 行之间的任意位置——都有可能被中断打断。一旦发生中断，操作系统就可能进行抢占式的线程调度，切换到另一个线程去执行。

这意味着：一个线程可能刚读取了全局变量的值（比如第 18 行），还没来得及完成后续计算和写回（比如第 23 行），就被切换出去了；而另一个线程此时也可能读取同一个变量，并基于旧值进行修改。最终两个线程写回的结果都可能是错误的。

由于多个线程在执行这段汇编代码（第 18~23 行）时，会同时访问同一个全局变量，从而引发数据不一致的问题，这种现象称为“竞争状态”（race condition）。

因此，我们将这段访问共享变量的代码片段称为临界区（critical section）。
临界区是指程序中访问共享资源（如全局变量、文件、设备等）的一段代码，它不能被多个线程同时执行，必须通过某种机制保证互斥访问——即同一时刻，最多只能有一个线程进入临界区。

下面是有两个线程T0、T1在一个时间段内的一种可能的执行情况：

时间	T0	T1	OS	共享变量a	寄存器a5
1	L18	-	-	0	a的高位地址
2	-	-	切换	0	0
3	-	L18	-	0	a的高位地址
4	L20	-	-	0	1
5	-	-	切换	0	a的高位地址
6	-	L20	-	0	1
7	-	-	切换	0	1
8	L23	-	-	1	1
9	-	-	切换	1	1
10	-	L23	-	1	1

一般情况下，线程 T0 执行完毕后，再执行线程 T1，那么共享全局变量 a 的值为 2。但在上面的执行过程中，可以看到在线程执行指令的过程中会发生线程切换，这样在时刻 10 的时候，共享全局变量 a 的值为 1，这不是我们预期的结果。

这里说的“不是预期结果”，是指我们本希望 a 被加了两次（T0 和 T1 各加一次），所以应该是 2，但实际只变成了 1。

出现这种情况的原因是：两个线程在操作系统的调度下（在哪个时刻调度具有不确定性），交错执行 $a = a + 1$ 的不同汇编指令序列，导致虽然增加全局变量 a 的代码被执行了两次，但结果还是只增加了 1。

这种多线程的最终执行结果不确定（indeterminate），取决于由于调度导致的、不确定指令执行序列的情况，就是竞态条件（race condition）。

如果每个线程在执行 $a = a + 1$ 这个 C 语句所对应的多条汇编语句过程中，不会被操作系统切换，那么就不会出现多个线程交叉读写全局变量的情况，也就不会出现结果不确定的问题了。

所以，访问（特指写操作）共享变量的代码片段，不能由多个线程同时执行（即并行），或者在一个时间段内都去执行（即并发）。要做到这一点，需要互斥机制的保障。

所谓“互斥机制”，就是确保同一时间只有一个线程能进入这段关键代码。

从某种角度上看，这种互斥性也是一种原子性，即线程在临界区的执行过程中，不会出现只执行了一部分，就被打断并切换到其他线程执行的情况。也就是说，要么线程执行的这一系列操作/指令都完成，要么这一系列操作/指令都不做，不会出现指令序列执行中被打断的情况。

锁的基本思路

要保证多线程并发执行中的临界区的代码具有互斥性或原子性，我们可以建立一种锁，只有拿到锁的线程才能在临界区中执行。这里的锁与现实生活中的锁的含义很类似。比如，我们可以写出如下的伪代码：

```
1 lock(mutex);    // 尝试取锁
2 a = a + 1;       // 临界区，访问临界资源 a
3 unlock(mutex);  // 释放锁
4 ...             // 剩余区
```

对于一个应用程序而言，它的执行是受到其执行环境的管理和限制的，而执行环境的主要组成就是用户态的系统库、操作系统和更底层的处理器。这说明我们需要有硬件和操作系统来对互斥进行支持。

一个自然的想法是：这个 lock/unlock 互斥操作就是 CPU 提供的机器指令，那上面这一段程序就很容易在计算机上行了。

这个想法看似合理，但存在局限性：CPU 的原子指令（如 test-and-set、compare-and-swap）虽然能保证对“锁变量”本身的读写是原子的，却无法直接识别或保护临界区中涉及的多个共享变量。

但需要注意，这里互斥的对象是线程的临界区代码，而临界区代码可以访问各种共享变量（简称临界资源）。只靠两条机器指令，难以识别各种共享变量，不太可能约束可能在临界区的各种指令执行共享变量操作的互斥性。

所以，我们还是需要有一些相对更灵活和复杂一点的方法，能够设置一种所有线程都能看到的标记。在一个能进入临界区的线程设置好这个标记后，其他线程都不能再进入临界区了。

总体上看，对临界区的访问过程分为四个部分：

1. 尝试取锁：查看锁是否可用，即临界区是否可访问（看占用临界区标志是否被设置）。如果可以访问，则设置占用临界区标志（锁不可用），并转到步骤 2；否则，线程忙等或被阻塞；
2. 临界区：访问临界资源的系列操作；
3. 释放锁：清除占用临界区标志（锁可用）。如果有线程被阻塞，会唤醒阻塞线程；
4. 剩余区：与临界区不相关部分的代码。

根据上面的步骤，可以看到锁机制有两种：让线程忙等的忙等锁（spin lock），以及让线程阻塞的睡眠锁（sleep lock）。

忙等锁会让线程在等待时持续检查锁状态，消耗 CPU 资源；睡眠锁则会让线程放弃 CPU，进入等待队列，由操作系统调度唤醒，适合等待时间较长的场景。

锁的实现大体上基于三类机制：用户态软件、机器指令硬件、内核态操作系统。下面我们介绍 rCore 中基于内核态操作系统级方法实现的支持互斥的锁。

我们还需要知道如何评价各种锁实现的效果。一般我们需要关注锁的三种属性：

1. 互斥性（mutual exclusion）：即锁是否能够有效阻止多个线程进入临界区，这是最基本的属性；
2. 公平性（fairness）：当锁可用时，每个竞争线程是否有公平的机会抢到锁；
3. 性能（performance）：即使用锁的时间开销，包括获取锁、释放锁以及等待锁所花费的总时间。

内核态操作系统级方法实现锁 — mutex 系统调用

使用 mutex 系统调用

如何能够实现轻量的可睡眠锁？一个自然的想法就是，让等待锁的线程进入睡眠状态（即不再占用 CPU），而由释放锁的线程显式地唤醒一个或多个正在等待该锁的线程。

这样做可以避免忙等浪费 CPU 资源，尤其适合临界区执行时间较长、或者锁竞争较激烈的场景。

如果有多个等待锁的线程，操作系统可以选择全部唤醒，让它们再次竞争锁；也可以只唤醒最早开始等待的那个线程（即遵循“先来先服务”的原则）。无论采用哪种策略，都需要操作系统的额外支持——特别是一个等待队列，用来记录当前有哪些线程在等待这把锁。

我们先看看多线程应用程序如何使用 mutex 系统调用的：

```
1// user/src/ch8b_mut_race.c
2...
3int mutex_id;
4int a;
5int threads[thread_count];
6void fun(long i)
7{
8    int t = i + 1;
9    for (int i = 0; i < per_thread; i++) {
10        assert_eq(mutex_lock(mutex_id), 0);
11        int old_a = a;
12        for (int i = 0; i < 500; i++) {
13            t = t * t % 10007;
14        }
15        a = old_a + 1;
16        assert_eq(mutex_unlock(mutex_id), 0);
17    }
18    exit(t);
19}
20
21int main()
22{
23    int64 start = get_mtime();
24    assert((mutex_id = mutex_blocking_create()) >= 0);
25    for (int i = 0; i < thread_count; i++) {
26        threads[i] = thread_create(fun, (void *)i);
27        assert(threads[i] > 0);
28    }
29    ...
30}
31
32// usr/lib/syscall.c
33int mutex_create()
34{
35    return syscall(SYS_mutex_create, 0);
36}
37
38int mutex_blocking_create()
```

```

39{
40 return syscall(SYS_mutex_create, 1);
41}
42
43int mutex_lock(int mid)
44{
45 return syscall(SYS_mutex_lock, mid);
46}
47
48int mutex_unlock(int mid)
49{
50 return syscall(SYS_mutex_unlock, mid);
51}

```

1. 第24行，创建了一个ID为 `mutex_id` 的互斥锁，对应的是第35行 `SYS_mutex_create` 系统调用；
2. 第10行，尝试获取锁（对应的是第45行 `SYS_mutex_lock` 系统调用），如果取得锁，将继续向下执行临界区代码；如果没有取得锁，将阻塞；
3. 第16行，释放锁（对应的是第50行 `SYS_mutex_unlock` 系统调用），如果有等待在该锁上的线程，则唤醒这些等待线程。

mutex 系统调用的实现

操作系统如何实现这些系统调用呢？首先考虑一下与此相关的核心数据结构，然后考虑与这些数据结构相关的函数或方法的实现。

注解

互斥锁根据在 lock 一个已被占用的锁时的行为，可分为阻塞互斥锁（blocking mutex）与自旋互斥锁（spinning mutex）。下文我们主要聚焦于阻塞互斥锁，但 ucore 中两者都通过同一个结构体 `struct mutex` 以及几个函数中的不同分支来实现。

在线程的视角中，互斥是一种每个线程都能看到的资源。而且在一个进程中，可以存在多个不同的互斥资源。因此，我们可以把所有的互斥资源集中起来，由进程统一管理，如下面代码第 4 行所示。

这里的关键点是：互斥锁不是全局唯一的，一个程序可以创建和使用多个互斥锁，分别保护不同的共享资源。

这里需要注意的是：`struct mutex mutex_pool[LOCK_POOL_SIZE]` 表示的是一个用于存放互斥锁实例的“内存池”，也就是多个互斥资源的集合。而 `struct mutex` 是内核中用来表示一个互斥锁的数据结构，它就是我们所说的“互斥资源”或“互斥锁”本身。

操作系统需要显式地施加某种控制机制，来决定当一个线程释放锁时，哪一个等待中的线程能够获得这把锁。为了做到这一点，操作系统必须维护一个等待队列，用来记录当前有哪些线程正在等待这把锁，如下面代码第 10 行所示。

等待队列通常是一个链表或队列结构，保存了所有因请求已被占用的锁而被阻塞的线程控制块（TCB）。当锁被释放时，操作系统会从该队列中选择一个线程（例如队首线程）将其唤醒，使其重新参与调度。

```

1struct proc {
2    ...
3    uint next_mutex_id;
4    struct mutex mutex_pool[LOCK_POOL_SIZE];
5};
6
7struct mutex {
8    uint blocking;
9    uint locked;
10   struct queue wait_queue;
11   // 等待队列的"分配"数据
12   int _wait_queue_data[WAIT_QUEUE_MAX_LENGTH];
13};

```

这样，在操作系统中，需要设计实现几个核心成员变量。互斥锁的成员变量有四个：表示是阻塞锁还是自旋锁的 `blocking`，是否锁上的 `locked`，和（阻塞锁会用到的）管理等待线程的等待队列 `wait_queue` 及其内存单元 `_wait_queue_data`；进程的成员变量：锁内存池 `mutex_pool` 及记录互斥锁分配情况的 `next_mutex_id`。

首先需要创建一个互斥锁，下面是应对 `SYSCALL_MUTEX_CREATE` 系统调用的创建互斥锁的函数：

```

1// os/syscall.c
2int sys_mutex_create(int blocking)
3{
4    struct mutex *m = mutex_create(blocking);
5    if (m == NULL) {
6        return -1;
7    }
8    int mutex_id = m - curr_proc()->mutex_pool;
9    return mutex_id;
10}
11
12// os/sync.c
13struct mutex *mutex_create(int blocking)
14{
15    struct proc *p = curr_proc();
16    if (p->next_mutex_id >= LOCK_POOL_SIZE) {
17        return NULL;
18    }
19    struct mutex *m = &p->mutex_pool[p->next_mutex_id];
20    p->next_mutex_id++;
21    m->blocking = blocking;
22    m->locked = 0;
23    if (blocking) {
24        // 阻塞锁需要等待队列，自旋锁不需要
25        init_queue(&m->wait_queue, WAIT_QUEUE_MAX_LENGTH,
26                m->_wait_queue_data);
27    }
28    return m;
29}

```

1. 第16~19行，互斥锁池还没用完，就找下一个可行的锁，否则返回NULL。

有了互斥锁，接下来就是实现 Mutex 的内核函数：对应 SYSCALL_MUTEX_LOCK 系统调用的 sys_mutex_lock。操作系统主要工作是，在锁已被其他线程获取的情况下，把当前线程放到等待队列中，并调度一个新线程执行。主要代码如下：

```
1// os/syscall.c
2int sys_mutex_lock(int mutex_id)
3{
4    if (mutex_id < 0 || mutex_id >= curr_proc()->next_mutex_id) {
5        return -1;
6    }
7    mutex_lock(&curr_proc()->mutex_pool[mutex_id]);
8    return 0;
9}
10
11// os/sync.c
12void mutex_lock(struct mutex *m)
13{
14    if (!m->locked) {
15        m->locked = 1;
16        return;
17    }
18    if (!m->blocking) {
19        ...
20    }
21    // 阻塞锁将在队列中等待
22    struct thread *t = curr_thread();
23    push_queue(&m->wait_queue, task_to_id(t));
24    // 别忘了将线程状态改为 SLEEPING
25    t->state = SLEEPING;
26    sched();
27    // 此时锁已释放 (locked = 1) 并传递给我，所以什么都不用做
28}
```

1. 第 7 行，以 ID 为 mutex_id 的互斥锁指针 m 为参数调用 mutex_lock 方法，具体工作由该方法来完成。
2. 第 18 行，分类讨论了自旋互斥锁的实现细节，这里我们聚焦阻塞互斥锁，省略了这一分支的细节；
3. 第 14 行，如果互斥锁 m 已经被其他线程获取了，那么在第 23 行，将把当前线程放入等待队列中；在第 25~26 行，让当前线程处于等待状态，并调度其他线程执行。
4. 第 15~16 行，如果互斥锁 m 还没被获取，那么当前线程会获取给互斥锁，并返回系统调用。

最后是实现 Mutex 的内核函数：对应 SYSCALL_MUTEX_UNLOCK 系统调用的 sys_mutex_unlock。操作系统的主要工作是，如果有等待在这个互斥锁上的线程，需要唤醒最早等待的线程。主要代码如下：

```
1// os/syscall.c
2int sys_mutex_unlock(int mutex_id)
3{
4    if (mutex_id < 0 || mutex_id >= curr_proc()->next_mutex_id) {
5        return -1;
6    }
7    mutex_unlock(&curr_proc()->mutex_pool[mutex_id]);
8    return 0;
9}
```

```

10
11// os/sync.c
12void mutex_unlock(struct mutex *m)
13{
14    if (m->blocking) {
15        struct thread *t = id_to_task(pop_queue(&m->wait_queue));
16        if (t == NULL) {
17            // 没有等待的线程，直接释放锁
18            m->locked = 0;
19        } else {
20            // 否则将锁交给下一个线程
21            t->state = RUNNABLE;
22            add_task(t);
23        }
24    } else {
25        m->locked = 0;
26    }
27}

```

1. 第 7 行，以 ID 为 `mutex_id` 的互斥锁 `m` 为参数调用 `mutex_unlock` 方法，具体工作由该方法来完成的。
2. 第 18 行，如果等待队列为空，直接释放锁。
3. 第 21-22 行，如果等待队列非空，则唤醒等待最久的线程 `t`，注意这里不改变 `m->locked`，可以想想为什么（实验答题之一）。

信号量机制

本节导读

在上一节中，我们介绍了互斥锁（mutex 或 lock）的起因、使用和实现过程。通过互斥锁，可以让线程在临界区执行时，独占临界资源。当我们需要更灵活的互斥访问或同步操作方式，比如最多只允许 N 个线程访问某个临界资源，或者让某个线程等待另一个线程执行完后再继续执行等场景时，互斥锁就显得不够用了。

在本节中，我们将介绍一种功能更强大、更灵活的同步互斥机制——信号量（Semaphore），包括它的设计思路、使用方法以及在操作系统中的具体实现。你会发现，信号量的实现依赖于互斥锁和处理器提供的原子指令，它是一种更高级的同步互斥机制。

提示：信号量并不是取代互斥锁，而是在其基础上扩展出更多控制能力。理解互斥锁是掌握信号量的前提。

信号量的起源和基本思路

1963 年前后，计算机科学家 Edsger Dijkstra 和他的团队在为 Electrologica X8 计算机开发操作系统（称为 THE 多道程序系统）时，提出了“信号量”（Semaphore）这一概念。信号量本质上是一种变量或抽象数据类型，用于协调多个线程对共享资源的访问。

信号量可以看作是对互斥锁的一种巧妙扩展。回想一下，互斥锁通常用一个初始值为 1 的整型变量表示：1 表示临界区空闲，0 表示已被占用，通过 lock/unlock 操作确保每次只有一个线程进入临界区。而信号量的初始值可以设为任意整数 N ：

- 如果 $N > 0$ ，表示最多允许 N 个线程同时进入临界区；
- 如果 $N \leq 0$ ，表示当前不允许任何线程进入，必须等到其他线程执行特定操作使信号量值增加后，才能唤醒等待的线程。

Dijkstra 为信号量定义了两种基本操作：P 操作和 V 操作。

1. P 操作 (Proberen, 荷兰语意为“尝试”) :

- 首先检查信号量的值是否大于 0;
- 如果大于 0, 则将其减 1, 并允许线程继续执行 (相当于成功获取资源) ;
- 如果等于 0, 则当前线程会被阻塞 (进入睡眠状态) , 并加入该信号量的等待队列中。

注意: 整个 P 操作 (包括检查、修改信号量值、可能的睡眠) 必须是**原子的**, 即不可被其他线程打断。这是因为信号量本身也是一种临界资源, 必须防止多个线程同时修改它导致错误。

2. V 操作 (Verhogen, 荷兰语意为“增加”) :

- 将信号量的值加 1;
- 然后检查是否有线程在该信号量上等待;
- 如果有, 就唤醒其中一个等待线程, 让它完成之前未完成的 P 操作;
- 如果没有等待线程, 则直接返回。

注意: V 操作也必须是原子的, 但与 P 操作不同, 执行 V 操作的线程**永远不会被阻塞**。

根据取值范围的不同, 信号量分为两类:

1. 计数信号量 (Counting Semaphore 或 General Semaphore) : 信号量的值可以是任意非负整数, 用于控制最多 N 个线程同时访问某类资源。
2. 二值信号量 (Binary Semaphore) : 信号量的值只能是 0 或 1, 功能上等价于互斥锁。

关键点: 互斥锁其实是信号量的一个特例——即初始值为 1 的二值信号量。因此, 信号量不仅能实现互斥, 还能实现更复杂的同步需求, 比如限制并发数量、线程间协作等。

信号量的一种实现伪代码如下所示:

```
1 void P(S) {
2     if (S >= 1)
3         S = S - 1;
4     else
5         <block and enqueue the thread>;
6 }
7 void V(S) {
8     if <some threads are blocked on the queue>
9         <unblock a thread>;
10    else
11        S = S + 1;
12 }
```

在上述实现中, S 的取值范围为大于等于 0 的整数。S 的初值一般设置为一个大于 0 的正整数, 表示可以进入临界区的线程数。当 S 取值为 1, 表示是二值信号量, 也就是互斥锁了。使用信号量实现线程互斥访问临界区的伪代码如下:

```

1static struct semaphore S = {1};
2
3// 线程 i
4void foo() {
5    ...
6    P(S);
7    execute Cricital Section;
8    V(S);
9    ...
10}

```

下面是另外一种信号量实现的伪代码：

```

1void P(S) {
2    S = S - 1;
3    if (S < 0)
4        <block and enqueue the thread>;
5}
6
7void V(S) {
8    S = S + 1;
9    if <some threads are blocked on the queue>
10        <unblock a thread>;
11}

```

在这种实现中，信号量 S 的初始值通常设为一个大于 0 的正整数，表示当前允许进入临界区的线程数量。但需要注意的是， S 的值在运行过程中可能变为负数：当有线程因执行 P 操作而被阻塞时， S 的值会继续递减，其绝对值就表示当前正在等待进入临界区的睡眠线程数量。

提示：虽然初值一般为非负数，但在 P/V 操作过程中，信号量的值可以小于 0。这是设计上的正常行为，并非错误状态。

信号量的另一种重要用途是实现线程间的同步（synchronization）。例如，我们可以将信号量的初始值设为 0：

- 当线程 A 对该信号量执行 P 操作时，由于信号量值为 0，线程 A 会立即被阻塞并进入睡眠状态；
- 之后，当另一个线程 B 执行 V 操作时，信号量值加 1，并唤醒线程 A；
- 此时，线程 A 被唤醒并完成 P 操作，继续执行后续代码。

通过这种方式，线程 B 中 V 操作之前的代码段（记为 B-stmts）和线程 A 中 P 操作之后的代码段（记为 A-stmts）之间就建立了一种明确的执行顺序关系：B-stmts 一定在 A-stmts 之前完成。

举例说明：这就像“发令枪”机制——线程 B 完成准备工作后“开枪”（执行 V），线程 A 听到枪声（被唤醒）才开始行动。这种模式常用于任务依赖、生产者-消费者等场景。

相关伪代码如下所示：

```

1static struct semaphore S = {1};
2//线程A
3...
4P(S);
5Label_2:
6A-stmts after Thread B::Label_1;

```

```
7...
8
9//线程B
10...
11B-stmts before Thread A::Label_2;
12Label_1:
13V(S);
14...
```

实现信号量

使用 semaphore 系统调用

我们通过一个例子来了解如何在程序中实际使用信号量。下面展示的是应用程序中对信号量系统调用的简单用法。可以看到，它的使用方式和上一节介绍的互斥锁系统调用很相似。

在这个例子中：

1. 主线程首先创建一个初值为 0 的信号量，名为 SEM_SYNC；
2. 接着，主线程创建两个线程：First 和 Second；
3. 线程 First 会先睡眠 10 毫秒；
4. 当线程 Second 开始执行时，它会尝试对信号量执行 P 操作（即申请资源），但由于信号量当前值为 0，因此线程 Second 会被阻塞并进入等待状态；
5. 当线程 First 睡醒后，会对同一个信号量执行 V 操作（即释放资源），将信号量的值加 1；
6. 这个 V 操作会唤醒正在等待的线程 Second，使其继续执行。

信号量的 P 操作（也叫 wait）会尝试减少信号量的值，如果值小于 0，则当前线程会被挂起；V 操作（也叫 signal）会增加信号量的值，并可能唤醒一个被 P 操作阻塞的线程。

通过这种方式，线程 First 和线程 Second 就建立了一种稳定的同步关系：Second 总是会在 First 完成某些操作之后才继续执行。

```
1const int SEM_SYNC = 0; //信号量ID
2void first() {
3    sleep(10);
4    puts("First work and wakeup Second");
5    semaphore_up(SEM_SYNC); //信号量V操作
6    exit(0);
7}
8void second() {
9    puts("Second want to continue,but need to wait first");
10    semaphore_down(SEM_SYNC); //信号量P操作
11    puts("Second can work now");
12    exit(0);
13}
14int main() {
15    // 创建信号量
16    assert_eq(semaphore_create(0), SEM_SYNC); // 信号量初值为0
17    // 创建 first, second 线程
18    ...
19}
```

```

20
21int semaphore_create(int res_count)
22{
23    return syscall(SYS_semaphore_create, res_count);
24}
25
26int semaphore_up(int sid)
27{
28    return syscall(SYS_semaphore_up, sid);
29}
30
31int semaphore_down(int sid)
32{
33    return syscall(SYS_semaphore_down, sid);
34}

```

1. 第 16 行，创建了一个初值为 0，ID 为 SEM_SYNC 的信号量，对应的是第 23 行 SYS_semaphore_create 系统调用；
2. 第 10 行，线程 Second 执行信号量 P 操作（对应第 33 行 SYS_semaphore_down 系统调用），由于信号量初值为 0，该线程将阻塞；
3. 第 5 行，线程 First 执行信号量 V 操作（对应第 28 行 SYS_semaphore_up 系统调用），会唤醒等待该信号量的线程 Second。

实现 semaphore 系统调用

操作系统是如何实现信号量系统调用的呢？我们仍然采用常见的分析方法：数据结构 + 方法。也就是说，首先明确与信号量相关的核心数据结构，然后设计并实现操作这些数据结构的函数或方法。

在操作系统中，从线程的视角来看，信号量是一种所有线程都能看到的共享资源。而且，在一个进程中，可以同时存在多个不同的信号量。因此，我们可以将进程内所有的信号量集中管理。例如，在下面的代码第 4 行中：

```
struct semaphore semaphore_pool[LOCK_POOL_SIZE];
```

这里的 `semaphore_pool` 是一个信号量资源池，用于存放当前进程中所有可用的信号量对象。

`LOCK_POOL_SIZE` 定义了该池子的最大容量，也就是一个进程最多能创建多少个信号量。

其中，`struct semaphore` 是内核中表示信号量的数据结构，它通常包含两个关键部分：

1. 一个整数值（即信号量的当前计数值）；
2. 一个等待队列（用于记录因执行 P 操作而被阻塞的线程）。

为了保证对信号量的操作是安全且正确的，操作系统必须显式地控制线程在执行 P 操作和 V 操作时的行为：

- 当线程执行 P 操作（申请资源）时，如果信号量值大于 0，则直接减 1 并继续执行；如果值为 0 或更小，则该线程需要被挂起（睡眠），并加入到该信号量的等待队列中；
- 当线程执行 V 操作（释放资源）时，会将信号量值加 1，并尝试从等待队列中唤醒一个被阻塞的线程（如果有）。

在我们的实现中：

P 操作由 `semaphore_down` 函数实现；

V 操作由 `semaphore_up` 函数实现。

```

1 struct proc {
2     int pid; // 进程ID
3     uint next_semaphore_id;
4     struct semaphore semaphore_pool[LOCK_POOL_SIZE];
5     ...
6 };
7
8 struct semaphore {
9     int count;
10    struct queue wait_queue;
11    // 等待队列的"分配"数据
12    int _wait_queue_data[WAIT_QUEUE_MAX_LENGTH];
13 };
14
15 struct semaphore *semaphore_create(int count)
16 {
17     struct proc *p = curr_proc();
18     if (p->next_semaphore_id >= LOCK_POOL_SIZE) {
19         return NULL;
20     }
21     struct semaphore *s = &p->semaphore_pool[p->next_semaphore_id];
22     p->next_semaphore_id++;
23     s->count = count;
24     init_queue(&s->wait_queue, WAIT_QUEUE_MAX_LENGTH, s->_wait_queue_data);
25     return s;
26 }
27
28 void semaphore_up(struct semaphore *s)
29 {
30     s->count++;
31     if (s->count <= 0) {
32         // up后count <= 0表示等待队列非空
33         struct thread *t = id_to_task(pop_queue(&s->wait_queue));
34         t->state = RUNNABLE;
35         add_task(t);
36     }
37 }
38
39 void semaphore_down(struct semaphore *s)
40 {
41     s->count--;
42     if (s->count < 0) {
43         // s->count < 0 表示需要等待 (state=SLEEPING)
44         struct thread *t = curr_thread();
45         push_queue(&s->wait_queue, task_to_id(t));
46         t->state = SLEEPING;
47         sched();
48     }
49 }

```

首先是核心数据结构：

1. 第 4 行，进程控制块中管理的信号量列表。

2. 第 9~12 行，信号量的核心数据成员：信号量值和等待队列。

然后是重要的三个成员函数：

1. 第 15 行，创建信号量，信号量初值为参数 `res_count`，信号量池的使用原理同互斥锁。
2. 第 28 行，实现 V 操作的 `up` 函数，第 31 行，当信号量值小于等于 0 时，将从信号量的等待队列中弹出一个线程放入线程就绪队列。
3. 第 39 行，实现 P 操作的 `down` 函数，第 225 行，当信号量值小于 0 时，将把当前线程放入信号量的等待队列，设置当前线程为挂起状态并选择新线程执行。

条件变量机制

本节导读

到目前为止，我们已经学习了操作系统提供的互斥锁和信号量。但在实际编程中，如果使用不当，这两种机制很容易导致效率低下、竞态条件、死锁，甚至其他难以预料的问题。

举个例子：多个线程同时修改共享数据，如果没有正确加锁，结果可能每次运行都不一样，这就是“竞态条件”。

为了简化同步编程、减少出错，计算机科学家针对一类常见场景设计了一种更高层的同步原语——条件变量（Condition Variable）。

具体来说，在某些情况下，一个线程需要等待某个特定条件成立后，才能继续往下执行。

我们来看一个具体例子：

有两个线程，分别叫 `first` 和 `second`。

- 线程 `first` 会把全局变量 `A` 设置为 1；
- 线程 `second` 必须等到 `A != 0` 这个条件成立之后，才能继续执行。

用伪代码表示如下：

```
1static int A = 0;
2void first() {
3    A=1;
4    ...
5}
6
7void second() {
8    while (A==0) {
9        // 忙等或睡眠等待 A==1
10    };
11    //继续执行相关事务
12}
```

在上面的例子中，如果线程 `second` 先执行，会忙等在 `while` 循环中，在操作系统的调度下，线程 `first` 会执行并把 `A` 赋值为 1 后，然后线程 `second` 再次执行时，就会跳出 `while` 循环，进行接下来的工作。配合互斥锁，可以正确完成上述带条件的同步流程，如下面的伪代码所示：

```
1static int A = 0;
2void first() {
3    mutex.lock();
```

```

4   A=1;
5   mutex.unlock();
6   ...
7}
8
9void second() {
10  mutex.lock();
11  while A == 0 {
12      mutex.unlock();
13      // 给其他线程加锁的机会
14      mutex.lock();
15  }
16  mutex.unlock();
17      //继续执行相关事务
18}

```

这种实现能执行，但效率低下，因为线程 second 会忙等检查，浪费处理器时间。我们希望有某种方式让线程 second 休眠，直到等待的条件满足，再继续执行。于是，我们可以写出如下的代码：

```

1static int A = 0;
2void first() {
3    mutex.lock();
4    A=1;
5    wakeup(second);
6    mutex.unlock();
7    ...
8}
9
10void second() {
11    mutex.lock();
12    while (A==0) {
13        wait();
14    };
15    mutex.unlock();
16    //继续执行相关事务
17}

```

粗略地看，这样似乎就能实现“睡眠等待”了。但请同学们仔细思考一下：当线程 second 进入睡眠时，它是否已经持有了 mutex 锁？

答案是肯定的——线程 second 是在持有 mutex 锁的状态下进入等待（睡眠）的。

如果两个线程的调度顺序是：先运行 second，再运行 first，那么就会出现以下情况：

- 线程 second 发现 $A == 0$ ，于是带着 mutex 锁进入睡眠；
- 线程 first 想修改 A，但因为 mutex 被 second 占着，自己也必须等待锁；
- 结果：两个线程都睡着了，谁也无法继续执行——这就是典型的死锁。

要解决这个问题，关键在于两个方面：

1. 如何让线程安全地等待某个条件成立？
2. 当条件变为真时，如何通知（唤醒）正在等待的线程？

针对这些问题，计算机科学家提出了 管程（Monitor）和 条件变量（Condition Variables）的机制。接下来，我们将深入讲解条件变量的设计思路与实现方式。

条件变量的基本思路

管程有一个核心特性：在任意时刻，最多只有一个线程可以执行管程中的过程。

这个特性天然保证了对共享变量的互斥访问——线程在调用管程内部函数时，无需额外担心其他线程同时修改数据。

管程通常是编程语言的一部分。编译器知道它的特殊性，因此在生成代码时，会自动插入必要的互斥逻辑（比如加锁/解锁），而不是依赖程序员手动处理。这样大大降低了出错概率。

虽然管程通过编译器简化了互斥的实现，但它本身还不足以支持复杂的线程协作。我们还需要一种线程间的通信机制，主要包括：

1. 等待机制：当线程在管程中发现某个条件不满足时，应能安全地阻塞自己，等待条件成立；
2. 唤醒机制：另一个线程在管程中将条件设为真后，应能通知（唤醒）正在等待的线程。

为了保证管程中始终只有一个活跃线程，唤醒操作的行为需要遵循特定规则。目前有三种经典语义方案：

1. Hoare 语义：发出唤醒信号的线程立即阻塞自己，让被唤醒的线程马上运行。（此时唤醒线程仍处于管程内部）
2. Hansen 语义：唤醒操作必须是管程过程中的最后一条语句，执行完后该线程立即退出管程。（唤醒线程离开管程）
3. Mesa 语义：发出唤醒信号的线程继续执行，只有当它完全退出管程后，被唤醒的线程才被允许运行。（唤醒线程仍在管程中）

实践中，大多数系统（如 POSIX 线程库）采用的是 Mesa 语义，因为它实现简单、性能较好，尽管它要求被唤醒的线程重新检查条件（因为条件可能在唤醒后又被改变）。

这种通信机制的具体实现，就是条件变量及其两个基本操作：`wait` 和 `signal`。

- 条件变量本质上是一个等待队列，用于存放因条件不满足而阻塞的线程；
- 当某线程发现条件不成立，就调用 `wait`，自动释放关联的互斥锁并进入睡眠；
- 另一个线程在使条件成立后，调用 `signal`（或 `broadcast`）来唤醒一个（或所有）等待线程。

注意：`wait` 操作必须与一个互斥锁配合使用。调用 `wait` 时，会原子地释放锁并进入等待；当被唤醒后，会自动重新获取该锁，再从 `wait` 返回。

早期的管程是在 Concurrent Pascal 等语言中直接支持的。但像 C 或 Rust 这样的语言，并没有内置管程。不过，我们可以通过手动组合互斥锁和条件变量，在这些语言中模拟出管程的行为。

事实上，现代 C 语言开发（例如使用 pthread 库）正是这样做的。借助互斥锁和条件变量，我们就可以正确实现前面提到的同步例子：

```
1static int A = 0;
2void first() {
3    mutex.lock();
4    A=1;
5    condvar.wakeup();
6    mutex.unlock();
7    ...
8}
9
```

```

10void second() {
11    mutex.lock();
12    while (A==0) {
13        condvar.wait(mutex); //在睡眠等待之前，需要释放mutex
14    };
15    mutex.unlock();
16    //继续执行相关事务
17}

```

有了上面的介绍，我们就可以实现条件变量的基本逻辑了。下面是条件变量的 wait 和 signal 操作的伪代码：

```

1void wait(mutex) {
2    mutex.unlock();
3    <block and enqueue the thread>;
4    mutex.lock();
5}
6
7void signal() {
8    <unblock a thread>;
9}

```

条件变量的 wait 操作实际上包含三个步骤：

1. 释放与之关联的互斥锁；
2. 将当前线程挂起（进入睡眠状态），并加入该条件变量的等待队列；
3. 当被其他线程唤醒后，重新尝试获取之前释放的互斥锁，成功后再从 wait 返回继续执行。

而条件变量的 signal 操作只做一件事：

查找挂在该条件变量上的等待线程（如果有的话），并唤醒其中一个（具体唤醒哪个取决于调度策略）。

注意：条件变量没有内部计数值，这一点和信号量完全不同。

信号量可以记录“还有多少资源可用”，但条件变量只是被动地传递“某个条件可能已成立”的通知。

因此，如果在没有任何线程等待时调用 signal，这个信号就会丢失——它不会被保存下来，后续的 wait 也无法感知到这次“空发”的信号。

实现条件变量

使用 condvar 系统调用

我们通过一个具体例子来看看如何在程序中使用条件变量。下面是一个面向应用程序的简单示例，其使用方式与上一节介绍的信号量系统调用类似。

在这个例子中：

- 主线程首先创建一个初始值为 1 的互斥锁（即处于未锁定状态）和一个条件变量；
- 然后创建两个子线程：First 和 Second；
- 线程 First 会先睡眠 10 毫秒；
- 与此同时，线程 Second 开始执行，发现全局变量 A 仍为 0（条件不满足），于是调用条件变量的 wait 操作，进入等待状态；
- 等到 10 毫秒后，线程 First 被唤醒，将 A 设置为 1，使线程 Second 所等待的条件成立；

- 接着，线程 First 调用条件变量的 `signal` 操作，唤醒正在等待的线程 Second；
- 线程 Second 被唤醒后，重新获得互斥锁，继续执行后续代码。

这样，线程 First 和线程 Second 就建立了一种稳定、安全的同步与互斥关系，避免了忙等待或死锁等问题。

```

1static int A = 0;    //全局变量
2
3const int CONDVAR_ID = 0;
4const int MUTEX_ID = 0;
5
6void first() {
7    sleep(10);
8    puts("First work, Change A --> 1 and wakeup Second");
9    mutex_lock(MUTEX_ID);
10   A = 1;
11   condvar_signal(CONDVAR_ID);
12   mutex_unlock(MUTEX_ID);
13   ...
14}
15void second() {
16   puts("Second want to continue,but need to wait A=1");
17   mutex_lock(MUTEX_ID);
18   while (A == 0) {
19       condvar_wait(CONDVAR_ID, MUTEX_ID);
20   }
21   mutex_unlock(MUTEX_ID);
22   ...
23}
24int main() {
25   // 创建条件变量和互斥锁
26   assert_eq(condvar_create(), CONDVAR_ID);
27   assert_eq(mutex_blocking_create(), MUTEX_ID);
28   // 创建 first, second 线程
29   ...
30}
31
32int condvar_create()
33{
34   return syscall(SYS_condvar_create);
35}
36
37int condvar_signal(int cid)
38{
39   return syscall(SYS_condvar_signal, cid);
40}
41
42int condvar_wait(int cid, int mid)
43{
44   return syscall(SYS_condvar_wait, cid, mid);
45}

```

1. 第 26 行，创建了一个 ID 为 CONDVAR_ID 的条件量，对应第 34 行 SYSCALL_CONDVAR_CREATE 系统调用；

2. 第 19 行，线程 Second 执行条件变量 wait 操作（对应第 44 行 SYSCALL_CONDVAR_WAIT 系统调用），该线程将释放 mutex 锁并阻塞；
3. 第 5 行，线程 First 执行条件变量 signal 操作（对应第 39 行 SYSCALL_CONDVAR_SIGNAL 系统调用），会唤醒等待该条件变量的线程 Second。

实现 condvar 系统调用

操作系统是如何实现条件变量系统调用的呢？

在应用程序线程看来，条件变量是一种进程内所有线程可见的共享资源。而且，在一个进程中，可以同时存在多个不同的条件变量。因此，操作系统通常会将这些条件变量集中管理。

例如，在内核或运行时系统中，可以用一个列表来统一维护当前进程创建的所有条件变量资源。

如下面代码第 9 行所示：

```
condvar_list: Vec<Option<Arc<Condvar>>>
```

这行代码表示：

1. `condvar_list` 是一个向量（动态数组），用于存储本进程中所有的条件变量；
2. 每个元素是一个 `Option<Arc<Condvar>>`，其中 `Arc` 表示该条件变量可以被多个线程安全地共享引用；
3. `Condvar` 是操作系统内核中代表条件变量的数据结构，其核心组成部分是一个等待队列，用于存放因调用 `wait` 而阻塞的线程。

当线程执行条件变量的 `wait` 或 `signal` 操作时，操作系统必须明确控制线程的睡眠与唤醒行为：

1. `wait` 操作由 `Condvar` 结构体的 `wait` 方法实现：它会将当前线程加入该条件变量的等待队列，并使其进入阻塞状态；
2. `signal` 操作由 `Condvar` 的 `signal` 方法实现：它会从等待队列中取出一个线程（如果有的话），并将其标记为可运行状态，交由调度器后续唤醒。

注意：这些操作必须与互斥锁紧密配合。例如，`wait` 必须在持有对应互斥锁的前提下调用，并在阻塞前自动释放锁，唤醒后再重新获取锁——整个过程需保证原子性，否则仍可能引发竞态条件。

```
1// os/proc.h
2struct proc {
3    int pid; // 进程ID
4    uint next_condvar_id;
5    struct condvar condvar_pool[LOCK_POOL_SIZE];
6    ...
7};
8
9// os/sync.h
10struct condvar {
11    struct queue wait_queue;
12    // 等待队列的"分配"数据
13    int _wait_queue_data[WAIT_QUEUE_MAX_LENGTH];
14};
15
16// os/sync.c
17struct condvar *condvar_create()
18{
19    struct proc *p = curr_proc();
```

```

20 if (p->next_condvar_id >= LOCK_POOL_SIZE) {
21     return NULL;
22 }
23 struct condvar *c = &p->condvar_pool[p->next_condvar_id];
24 p->next_condvar_id++;
25 init_queue(&c->wait_queue, WAIT_QUEUE_MAX_LENGTH, c->_wait_queue_data);
26 return c;
27}
28
29void cond_signal(struct condvar *cond)
30{
31     struct thread *t = id_to_task(pop_queue(&cond->wait_queue));
32     if (t) {
33         t->state = RUNNABLE;
34         add_task(t);
35     }
36}
37
38void cond_wait(struct condvar *cond, struct mutex *m)
39{
40     // 条件变量会先解锁互斥锁，返回时再重新加锁
41     mutex_unlock(m);
42     struct thread *t = curr_thread();
43     // 现在只需等待条件变量
44     push_queue(&cond->wait_queue, task_to_id(t));
45     t->state = SLEEPING;
46     sched();
47     mutex_lock(m);
48}

```

首先是核心数据结构：

1. 第 5 行，进程控制块中管理的条件变量列表。
2. 第 13 行，条件变量的核心数据成员：等待队列。

然后是重要的三个成员函数：

1. 第 17 行，创建条件变量，即创建了一个空的等待队列。
2. 第 29 行，实现 signal 操作，将从条件变量的等待队列中弹出一个线程放入线程就绪队列。
3. 第 38 行，实现 wait 操作，释放 m 互斥锁，将把当前线程放入条件变量的等待队列，设置当前线程为挂起状态并选择新线程执行。在恢复执行后，再加上 m 互斥锁。

编程作业：死锁检测

在理解了互斥锁、信号量和条件变量之后，我们会发现一个潜在的危险：如果多个线程以不同的顺序申请多个锁，就可能发生**死锁**。

任务目标

在现有同步原语基础上，实现基于**银行家算法**的死锁检测机制。当线程尝试获取锁时，系统应能检测是否会导致死锁，并在发现死锁时返回特殊错误码。

理解死锁

回顾前面提到的死锁概念：

死锁的四个必要条件：

1. **互斥**：资源一次只能被一个线程持有
2. **持有并等待**：线程持有资源的同时等待其他资源
3. **不可抢占**：资源只能由持有者主动释放
4. **循环等待**：存在线程的循环等待链

一个典型的死锁场景：

```
线程A：持有 mutex1，等待 mutex2
线程B：持有 mutex2，等待 mutex1
→ 两个线程永远等待，形成死锁
```

银行家算法思想

银行家算法用于**死锁检测/避免**，在分配资源前模拟"最好情况"——假设每个能运行的线程都会运行完毕并释放资源。如果这样都无法让所有线程完成，就存在死锁。

核心数据结构：

- `Available[m]`：系统当前可用的各类资源数量
- `Allocation[n][m]`：每个线程已分配的资源
- `Request[n][m]`：每个线程正在请求的资源

算法流程：

- ```
1. work = Available, Finish[i] = false (for all i)

2. 找到一个线程 i 满足：
 - Finish[i] == false
 - Request[i] <= work

3. 如果找到：
 - work = work + Allocation[i] // 假设 i 运行完毕释放资源
 - Finish[i] = true
 - 回到步骤 2

4. 如果所有 Finish[i] == true，则安全；否则死锁
```

## 第一步：扩展进程控制块

需要在进程中维护资源分配和请求矩阵：

```
// os/proc.h

struct proc {
 // ... 原有字段 ...

 // 同步原语池（已有）
 uint next_mutex_id, next_semaphore_id, next_condvar_id;
 struct mutex mutex_pool[LOCK_POOL_SIZE];
 struct semaphore semaphore_pool[LOCK_POOL_SIZE];
 struct condvar condvar_pool[LOCK_POOL_SIZE];

 /* ch8: 死锁检测相关 */
 int deadlock_detect_enable;
 int mutex_allocation[NTHREAD][LOCK_POOL_SIZE]; /* ch8: 分配矩阵 */
 int mutex_request[NTHREAD][LOCK_POOL_SIZE]; /* ch8: 请求矩阵 */
 int sem_allocation[NTHREAD][LOCK_POOL_SIZE];
 int sem_request[NTHREAD][LOCK_POOL_SIZE];
};
```

## 第二步：维护分配矩阵

在每次锁操作时更新矩阵：

**mutex\_lock** 中：

```
void mutex_lock(struct mutex *m)
{
 int id = m - curr_proc()->mutex_pool;
 struct proc *p = curr_proc();
 struct thread *t = curr_thread();

 if (!m->locked) {
 /* ch8: 直接获得锁，更新分配矩阵 */
 m->locked = 1;
 p->mutex_allocation[t->tid][id] = 1;
 return;
 }

 /* ch8: 需要等待，记录请求 */
 p->mutex_request[t->tid][id] = 1;

 // ... 等待逻辑 ...

 /* ch8: 获得锁后，更新矩阵 */
 p->mutex_request[t->tid][id] = 0;
 p->mutex_allocation[t->tid][id] = 1;
}
```

mutex\_unlock 中:

```
void mutex_unlock(struct mutex *m)
{
 int id = m - curr_proc()->mutex_pool;
 struct proc *p = curr_proc();

 /* ch8: 释放锁, 更新分配矩阵 */
 p->mutex_allocation[curr_thread()->tid][id] = 0;

 // ... 唤醒等待者逻辑 ...
}
```

### 第三步: 实现死锁检测算法

```
// os/syscall.c

/* ch8: 银行家算法检测死锁 */
int deadlock_detect(struct proc *p)
{
 int work_mutex[LOCK_POOL_SIZE];
 int work_sem[LOCK_POOL_SIZE];
 int finish[NTHREAD];
 int i, j;

 /* ch8: 初始化 work = Available */
 for (i = 0; i < LOCK_POOL_SIZE; i++) {
 /* ch8: mutex: 可用 = 未被锁定 */
 work_mutex[i] = p->mutex_pool[i].locked ? 0 : 1;
 /* ch8: semaphore: 可用 = count > 0 的部分 */
 work_sem[i] = p->semaphore_pool[i].count > 0 ?
 p->semaphore_pool[i].count : 0;
 }

 /* ch8: 初始化 Finish */
 for (i = 0; i < NTHREAD; i++) {
 /* ch8: 未使用或已退出的线程视为已完成 */
 finish[i] = (p->threads[i].state == T_UNUSED ||
 p->threads[i].state == EXITED) ? 1 : 0;
 }

 /* ch8: 银行家算法主循环 */
 while (1) {
 int found = 0;
 for (i = 0; i < NTHREAD; i++) {
 if (!finish[i]) {
 int possible = 1;

 /* ch8: 检查 mutex 需求: Request <= Work */
 for (j = 0; j < LOCK_POOL_SIZE; j++) {
 if (p->mutex_request[i][j] > work_mutex[j]) {
 possible = 0;
 }
 }
 }
 }
 }
}
```



```

 break;
 }
}
if (!possible) continue;

/* ch8: 检查 semaphore 需求 */
for (j = 0; j < LOCK_POOL_SIZE; j++) {
 if (p->sem_request[i][j] > work_sem[j]) {
 possible = 0;
 break;
 }
}

if (possible) {
 /* ch8: 假设线程 i 完成, 释放资源 */
 /* ch8: work = work + Allocation */
 for (j = 0; j < LOCK_POOL_SIZE; j++) {
 work_mutex[j] += p->mutex_allocation[i][j];
 work_sem[j] += p->sem_allocation[i][j];
 }
 finish[i] = 1;
 found = 1;
}
}
}
if (!found) break;
}

/* ch8: 检查是否所有线程都能完成 */
for (i = 0; i < NTHREAD; i++) {
 if (!finish[i])
 return 1; /* ch8: 发现死锁 */
}
return 0;
}

```

## 第四步：在锁操作中集成检测

```

// os/syscall.c

/* ch8: 带死锁检测的 mutex_lock */
int sys_mutex_lock(int mutex_id)
{
 if (mutex_id < 0 || mutex_id >= curr_proc()->next_mutex_id)
 return -1;

 struct proc *p = curr_proc();
 int tid = curr_thread()->tid;

 if (p->deadlock_detect_enable) {
 /* ch8: 先尝试标记请求 */
 p->mutex_request[tid][mutex_id] = 1;
 }
}

```

```

 /* ch8: 检测死锁 */
 if (deadlock_detect(p)) {
 /* ch8: 死锁! 撤销请求并返回错误 */
 p->mutex_request[tid][mutex_id] = 0;
 return -0xDEAD; /* ch8: 返回特殊错误码 */
 }

 /* ch8: 安全, 清除临时请求标记 */
 p->mutex_request[tid][mutex_id] = 0;
}

mutex_lock(&curr_proc()->mutex_pool[mutex_id]);
return 0;
}

/* ch8: 带死锁检测的 semaphore_down */
int sys_semaphore_down(int semaphore_id)
{
 if (semaphore_id < 0 || semaphore_id >= curr_proc()->next_semaphore_id)
 return -1;

 struct proc *p = curr_proc();
 int tid = curr_thread()->tid;

 if (p->deadlock_detect_enable) {
 p->sem_request[tid][semaphore_id] = 1;

 if (deadlock_detect(p)) {
 p->sem_request[tid][semaphore_id] = 0;
 return -0xDEAD;
 }

 p->sem_request[tid][semaphore_id] = 0;
 }

 semaphore_down(&curr_proc()->semaphore_pool[semaphore_id]);
 return 0;
}

```

## 第五步：添加控制系统调用

```

// os/syscall.c

/* ch8: 启用/禁用死锁检测 */
int sys_enable_deadlock_detect(int is_enable)
{
 struct proc *p = curr_proc();
 p->deadlock_detect_enable = is_enable;
 return 0;
}

```

## 系统调用接口

| 系统调用                                        | 功能              |
|---------------------------------------------|-----------------|
| <code>enable_deadlock_detect(enable)</code> | 启用(1)/禁用(0)死锁检测 |

返回值约定：

- 正常操作：返回 0
- 检测到死锁：返回 `-0xDEAD` (-57005)

## 分配矩阵维护总结

| 时机    | 操作                                                         |
|-------|------------------------------------------------------------|
| 尝试获取锁 | <code>request[tid][id] = 1</code>                          |
| 成功获取锁 | <code>request[tid][id] = 0, allocation[tid][id] = 1</code> |
| 释放锁   | <code>allocation[tid][id] = 0</code>                       |
| 检测到死锁 | 撤销请求，不执行实际操作                                               |

## 算法复杂度

银行家算法的时间复杂度为  $O(n^2m)$ ，其中：

- $n$  = 线程数 (NTHREAD)
- $m$  = 资源类型数 (LOCK\_POOL\_SIZE)

对于嵌入式操作系统中有限的线程和锁数量，这个开销是可接受的。

## 文件修改清单

| 文件                        | 修改内容                                                                                                                                                      |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>os/proc.h</code>    | 添加死锁检测矩阵和使能标志                                                                                                                                             |
| <code>os/proc.c</code>    | 初始化死锁检测相关字段                                                                                                                                               |
| <code>os/sync.c</code>    | 修改 <code>mutex_lock/unlock</code> 更新分配矩阵                                                                                                                  |
| <code>os/syscall.c</code> | 实现 <code>deadlock_detect()</code> 、修改 <code>sys_mutex_lock()</code> 和 <code>sys_semaphore_down()</code> 集成检测、添加 <code>sys_enable_deadlock_detect()</code> |